Experiment 5: Sentiment Analysis Using Naive Bayes

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

# Small demonstration dataset
data = {
    "comment": [
        "I love this product",
        "This is an excellent product",
        "The experience was wonderful",
        "I am very happy with the service",
        "The movie was fantastic",
        "This product is amazing",
        "I really enjoyed the experience",
        "The service was excellent",
        "I hate this product",
        "This is a terrible product",
        "The experience was awful",
        "I am very disappointed with the service",
        "The movie was boring",
        "This product is horrible",
        "I regret buying this product",
        "The service was very poor",
        "I dislike this experience",
        "The quality is bad"
    ],
    "sentiment": [
        "Positive", "Positive", "Positive", "Positive",
        "Positive", "Positive", "Positive", "Positive",
        "Negative", "Negative", "Negative", "Negative",
        "Negative", "Negative", "Negative", "Negative",
        "Negative", "Negative"
    ]
}

df = pd.DataFrame(data)
```

```
print("Dataset:")
print(df.head())
print("\nClass distribution:")
print(df["sentiment"].value_counts())
```

```
Dataset:
                          comment sentiment
0              I love this product  Positive
1     This is an excellent product  Positive
2     The experience was wonderful  Positive
3  I am very happy with the service  Positive
4           The movie was fantastic  Positive

Class distribution:
sentiment
Negative    10
Positive     8
Name: count, dtype: int64
```

```
X = df["comment"]
y = df["sentiment"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# Convert text to word-count vectors and train Naive Bayes
model = Pipeline([
    ("vectorizer", CountVectorizer()),
    ("classifier", MultinomialNB())
])

model.fit(X_train, y_train)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Model training completed.")
```

```
Training samples: 13
Testing samples: 5
Model training completed.
```

```
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.2%}")
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred, labels=["Negative", "Positive"]))

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    labels=["Negative", "Positive"],
    display_labels=["Negative", "Positive"],
    cmap="Blues"
)

plt.title("Naive Bayes Sentiment Classification")
plt.tight_layout()
plt.show()
```

```
Accuracy: 20.00%

Classification Report:
              precision    recall  f1-score   support

    Negative       0.33      0.33      0.33         3
```

```
new_comments = [
    "The product is wonderful",
    "The service was terrible"
]

predictions = model.predict(new_comments)

for comment, sentiment in zip(new_comments, predictions):
    print(f"Comment: {comment}")
    print(f"Predicted Sentiment: {sentiment}\n")
```

```
Comment: The product is wonderful
Predicted Sentiment: Negative

Comment: The service was terrible
Predicted Sentiment: Negative
```

